

Evaluacion de Robustez y Escalabilidad

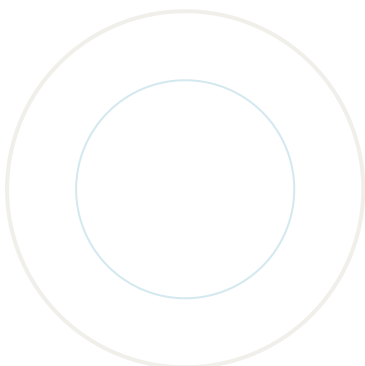
Plataforma: TiendApp

Tipo: SaaS Store Builder

Framework: Next.js 16 + Prisma + Zustand

Fecha: Abril 2026

Analisis exhaustivo de 49 archivos fuente que cubre seguridad, arquitectura, escalabilidad y calidad de componentes. Incluye plan de remediacion en 4 fases con priorizacion por impacto en el negocio.



6 CRITICOS

49 ARCHIVOS

Tabla de Contenidos

1. Resumen Ejecutivo	2
2. Vulnerabilidades Criticas de Seguridad	2
3. Problemas de Alta Prioridad	5
4. Analisis de Escalabilidad	7
5. Evaluacion de Componentes	8
6. Mapa de Endpoints API	9
7. Plan de Remediacion Priorizado	10
8. Conclusion	12

1. Resumen Ejecutivo

TiendApp es una plataforma SaaS tipo constructor de tiendas en linea dirigida al mercado peruano. Permite a pequenos y medianos negocios crear su propia tienda virtual en minutos, con integracion de WhatsApp para atencion al cliente, multiples plantillas de diseno, y un sistema de suscripcion con planes Free, Pro y Premium. La plataforma esta construida sobre Next.js 16 con App Router, Prisma ORM para acceso a base de datos, Zustand para gestion de estado en el cliente, y Tailwind CSS con shadcn/ui para la interfaz de usuario. El proyecto se despliega en Vercel con salida en modo standalone.

Se realizo una auditoria exhaustiva de 49 archivos fuente que abarca 15 rutas API, 3 componentes de autenticacion, 8 componentes de dashboard, 7 plantillas de tienda, 7 componentes de landing page, 6 componentes de administracion, 3 paginas informativas, y 5 archivos de logica central. El objetivo de esta auditoria es evaluar si la plataforma tiene la robustez, seguridad y escalabilidad necesarias para operar como un negocio real que alberga multiples tiendas de forma simultanea.

La conclusion principal es que TiendApp tiene una base funcional solida con componentes de interfaz bien disenados y una estructura de datos adecuada. Sin embargo, existen vulnerabilidades criticas de seguridad que deben resolverse antes de aceptar cualquier trafico de produccion real, ademas de problemas arquitectonicos que limitan su capacidad de escalar a cientos o miles de tiendas. A continuacion se presenta el resumen de hallazgos por categoria de severidad.

Categoría	Cantidad	Estado
Criticas (Bloqueantes)	6	Requieren accion inmediata
Altas (Importantes)	8	Resolver antes de lanzamiento
Medias (Mejoras)	10	Planificar para post-lanzamiento
Bajas (Polish)	5	Optimizaciones futuras

Tabla 1: Resumen de hallazgos por severidad

2. Vulnerabilidades Criticas de Seguridad

Esta seccion detalla las vulnerabilidades que representan un riesgo inmediato para la integridad del sistema y la seguridad de los datos de los usuarios. Cualquiera de estos problemas, si es explotada antes de ser corregida, podria comprometer toda la plataforma, incluyendo el acceso no autorizado a cuentas de administrador, la manipulacion de

suscripciones de pago, o el robo masivo de credenciales. Es imperativo que todas estas vulnerabilidades sean mitigadas antes de que la plataforma acepte cualquier cliente real o dato sensible de produccion.

2.1 Secreto JWT expuesto en codigo fuente

En el archivo `src/lib/auth.ts`, la linea 4 contiene un secreto JWT de respaldo codificado directamente en el codigo fuente. Si la variable de entorno `JWT_SECRET` no esta configurada, el sistema utiliza automaticamente el valor

"`tiendapp-secret-change-in-production-2024`" como secreto para firmar todos los tokens de autenticacion. Esto significa que cualquier persona que tenga acceso al codigo fuente (por ejemplo, a traves de un repositorio publico de GitHub o mediante la inspeccion del bundle de JavaScript del cliente) puede forjar tokens JWT validos, incluidos tokens con rol de administrador, otorgandose acceso completo al sistema. Esta es la vulnerabilidad de mayor prioridad en toda la plataforma.

La solucion correcta es eliminar completamente el valor de respaldo y hacer que la aplicacion falle de forma explicita si la variable de entorno no esta presente. De esta manera, un servidor sin configuracion adecuada nunca podra iniciar y firmar tokens con un secreto predecible.

2.2 Webhook de pagos sin verificacion de firma

El endpoint `PUT` de `/api/payments` esta disenado para recibir webhooks de pasarelas de pago como Culqi o Niubiz, pero no implementa absolutamente ninguna verificacion de firma criptografica. Cualquier usuario que envíe una solicitud `HTTP` con el cuerpo `JSON` adecuado, incluyendo un campo "status" con valor "succeeded" o "paid", puede activar suscripciones Premium de forma gratuita para cualquier cuenta de usuario. Dado que este endpoint esta completamente abierto sin autenticacion ni verificacion, representa un riesgo financiero directo: un atacante podria activar el plan mas costoso para todos los usuarios sin pagar un solo centavo, causando perdidas economicas significativas al negocio.

La correccion requiere implementar la verificacion de firma `HMAC` segun la documentacion oficial de la pasarela de pago seleccionada. Adicionalmente, el endpoint debe validar que la solicitud proviene de las direcciones `IP` conocidas del gateway de pago y registrar un log de auditoria para cada transaccion recibida.

2.3 Claves de Supabase expuestas en archivo .env

El archivo `.env` del proyecto contiene la clave de servicio (Service Role Key) de Supabase. Esta clave es equivalente a un acceso de superadministrador que permite leer, escribir y eliminar cualquier dato en la base de datos de Supabase, ignorando completamente las politicas de seguridad a nivel de fila (Row Level Security). Si este archivo se compromete, ya sea mediante un acceso no autorizado al servidor o porque fue commiteado accidentalmente

al repositorio de Git, un atacante tendría acceso total a todas las tiendas, usuarios, productos, pagos y datos personales almacenados en la plataforma. Es fundamental rotar inmediatamente esta clave y migrar todas las credenciales sensibles a un gestor de secretos como Vercel Environment Variables o AWS Secrets Manager.

2.4 Contraseñas en texto plano en datos de prueba

El archivo `src/lib/mock-data.ts` contiene cuatro usuarios de prueba con contraseñas en texto plano: "admin123", "demo123", "tienda123" y "belen123". Aunque estos datos están destinados a ser utilizados solo en entorno de desarrollo, el archivo es importado directamente por el store de Zustand y se incluye en el bundle de JavaScript que se envía al navegador del usuario final. Esto significa que cualquier visitante puede abrir las herramientas de desarrollo del navegador y encontrar estas credenciales, las cuales podrían ser utilizadas si los usuarios de prueba existen en la base de datos de producción. Además, el endpoint de autenticación en `src/app/api/auth/route.ts` contiene código que compara contraseñas en texto plano como mecanismo de migración, lo que indica que datos reales en la base de datos podrían almacenarse sin cifrar.

2.5 Ausencia de limitación de tasa en endpoints de autenticación

Los endpoints de login (`POST /api/auth`) y registro (`POST /api/users`) no implementan ningún mecanismo de limitación de tasa (rate limiting). Esto permite que un atacante automatice intentos de fuerza bruta para adivinar contraseñas sin restricción alguna, o que cree miles de cuentas falsas para saturar la base de datos y consumir recursos del servidor. En un escenario de negocio real con múltiples tiendas, la ausencia de rate limiting convierte a la plataforma en un blanco fácil para ataques de denegación de servicio (DoS) y ataques de fuerza bruta contra cuentas de administrador y propietarios de tiendas. La implementación de rate limiting con herramientas como Upstash Ratelimit o un middleware personalizado es esencial antes del lanzamiento.

2.6 Endpoint de seed (siembra) como GET sin protección adecuada

El endpoint `GET /api/seed` está protegido por autenticación de administrador, pero utiliza el método HTTP GET para una operación que ejecuta `deleteMany()` en todas las tablas de la base de datos antes de insertar datos de prueba. El uso del método GET es problemático porque los crawlers de buscadores, los servicios de pre-carga de CDN, y los logs de servidores pueden disparar accidentalmente este endpoint si un token de administrador válido es interceptado. Si esto ocurre, se eliminarían todos los datos de producción de todas las tiendas y se reemplazarían con datos de ejemplo. La corrección inmediata es cambiar este

endpoint al metodo POST, agregar un parametro de confirmacion, y en el futuro convertirlo en un script de linea de comandos en lugar de una ruta API accesible.

3. Problemas de Alta Prioridad

Los siguientes problemas no representan una amenaza inmediata como las vulnerabilidades criticas, pero son condiciones que deben resolverse antes de que la plataforma maneje multiples tiendas en produccion. Su presencia incrementa significativamente el riesgo de fallas operativas, perdida de datos y una experiencia de usuario deficiente a medida que el volumen de tiendas y usuarios crece.

3.1 Arquitectura SPA dentro de Next.js App Router

Toda la aplicacion funciona como un SPA (Single Page Application) dentro de Next.js. El archivo `src/app/page.tsx` simplemente renderiza el componente `AppRouter`, y toda la navegacion entre paginas se maneja a traves del store de Zustand con el campo "route", usando Framer Motion para transiciones. Esto significa que Next.js se utiliza unicamente como servidor de archivos estaticos, desaprovechando completamente sus capacidades de Server-Side Rendering (SSR), generacion estatica (SSG), y optimizacion de imagenes. El impacto mas significativo es que las paginas de las tiendas de los clientes no tendran SEO: los motores de busqueda como Google no pueden indexar los productos ni el contenido de las tiendas, lo cual es critico para el exito comercial de cualquier tienda online. Ademias, el tiempo de carga inicial es mayor porque el navegador debe descargar todo el JavaScript de la aplicacion antes de mostrar cualquier contenido.

3.2 SQLite como base de datos de produccion

El esquema de Prisma esta configurado para usar SQLite mediante `file:./local.db`. SQLite es una base de datos de archivo local excelente para desarrollo y prototipos, pero tiene limitaciones fundamentales para un sistema multi-tenant que alberga multiples tiendas: no soporta concurrencia de escritura (solo un escritor a la vez), carece de pool de conexiones, tiene un limite practico en el tamano de la base de datos, y no ofrece replicacion ni alta disponibilidad. Cuando multiples usuarios de diferentes tiendas realizan operaciones simultaneas (actualizar productos, procesar pagos, registrar visitas), SQLite puede generar bloqueos y errores de "database is locked". La migracion a PostgreSQL a traves de Supabase (para el cual ya existen variables de entorno configuradas) es una prioridad arquitectonica fundamental para la escalabilidad.

3.3 Almacenamiento de JWT en localStorage

Los tokens de autentificación JWT se almacenan en el `localStorage` del navegador con la clave `"tiendapp_token"`. Esta práctica es vulnerable a ataques de Cross-Site Scripting (XSS): si un atacante logra inyectar código JavaScript malicioso en cualquier parte de la aplicación, podría leer el token y robar la sesión del usuario. La alternativa recomendada es almacenar los JWT en cookies `httpOnly` que no son accesibles desde JavaScript. Además, la plataforma no implementa tokens de refresco (refresh tokens), lo que obliga a los usuarios a reiniciar sesión cada vez que el token expira (actualmente cada 7 días), y no hay mecanismo de invalidación de sesiones en el lado del servidor cuando un usuario cierra sesión o cambia su contraseña.

3.4 Settings almacenados en memoria volátil

El endpoint `/api/settings` almacena la configuración de la plataforma (nombre, modo mantenimiento, registros habilitados) en un objeto JavaScript dentro del archivo de la ruta. Cada vez que el servidor se reinicia (algo común en Vercel durante deployments o debido a la naturaleza serverless), toda la configuración se pierde y vuelve a los valores predeterminados. Esto significa que si un administrador activa el modo de mantenimiento o desactiva los registros, esos cambios desaparecerán con el próximo deploy. La solución es crear un modelo `PlatformSetting` en Prisma y persistir todos los ajustes en la base de datos.

3.5 Falta de validación de entrada con Zod

Ninguno de los endpoints API implementa validación de esquema con librerías como Zod. Los cuerpos de las solicitudes POST y PUT se parsean directamente con `request.json()` y se pasan a Prisma sin verificar la estructura, los tipos de datos, ni los rangos de valores. Esto permite que datos malformados, campos inesperados o valores peligrosos (como scripts HTML en descripciones de productos) lleguen a la base de datos. En un sistema multi-tenant, la inyección de contenido malicioso en descripciones de productos podría afectar a todos los visitantes de las tiendas. La implementación de schemas Zod para todos los endpoints es fundamental para la integridad de datos.

3.6 Errores TypeScript silenciados en build

El archivo `next.config.ts` tiene la opción `typescript.ignoreBuildErrors: true`, lo que significa que el proceso de build ignora completamente cualquier error de tipos de TypeScript. Esto oculta bugs potenciales que solo se manifestarán en tiempo de ejecución, cuando el usuario interactúe con la funcionalidad afectada. Además, la configuración de ESLint tiene más de 25 reglas desactivadas, eliminando una capa importante de detección de problemas de código. Para un proyecto que aspira a ser un negocio de producción con múltiples tiendas, la calidad del código debe ser garantizada por herramientas automáticas, no dependiente de revisiones manuales.

3.7 Flujo de recuperación de contraseña sin implementar

El componente `ResetPasswordPage.tsx` existe en la interfaz, pero la funcionalidad real no esta implementada. El formulario llama al endpoint `PUT /api/auth`, el cual no existe como ruta API, y la pagina procede independientemente de la respuesta. No hay generacion de tokens de restablecimiento, no hay envio de correos electronicos, y no hay verificacion de identidad. Para un negocio que maneja datos de multiples tiendas, la capacidad de recuperar contraseñas de forma segura es un requisito fundamental de seguridad y experiencia de usuario.

3.8 Falta de middleware de Next.js

El proyecto no tiene un archivo `middleware.ts` de Next.js. Este middleware es el punto central para implementar protecciones a nivel de edge como CSRF (Cross-Site Request Forgery), rate limiting global, redirecciones de rutas protegidas, y validacion de tokens antes de que la solicitud llegue a los endpoints API. Sin esta capa de seguridad intermedia, cada endpoint API debe implementar sus propias verificaciones de forma independiente, lo cual aumenta la probabilidad de olvidar alguna proteccion en un endpoint especifico.

4. Analisis de Escalabilidad

Esta seccion evalua la capacidad de la plataforma para crecer de forma controlada y manejar un volumen significativo de tiendas, productos y usuarios simultaneos sin degradacion del servicio.

4.1 Gestion dual de estado (Mock + API)

El store de Zustand inicializa su estado con datos de prueba (`MOCK_USERS`, `MOCK_STORES`, `MOCK_PRODUCTS`) y simultaneamente ejecuta `syncFromAPI()` para obtener datos reales del servidor. Este patron de doble estado genera condiciones de carrera donde la interfaz muestra brevemente datos falsos antes de reemplazarlos con datos reales. A medida que crece el numero de tiendas y productos, la sincronizacion se vuelve mas lenta y el parpadeo de datos se hace mas evidente para el usuario. Ademias, si la sincronizacion falla, el usuario ve datos obsoletos sin ninguna indicacion de error. La arquitectura correcta deberia mostrar un estado de carga mientras se obtienen datos del servidor, y nunca mezclar datos de prueba con datos de produccion.

4.2 Eliminacion de productos sin persistencia al servidor

La funcion `deleteProduct` en el store de Zustand solo elimina el producto del estado local, pero no realiza ninguna llamada API para persistir la eliminacion en la base de datos. Esto significa que cuando el usuario refresca la pagina o cierra el navegador, el producto eliminado reaparece. En un escenario con multiples tiendas, un propietario que elimine un producto

quedaria con la impresion de que la operacion fue exitosa, cuando en realidad el producto sigue visible para sus clientes. Esta inconsistencia entre el estado local y el servidor es un problema critico de confiabilidad que afecta directamente la confianza del usuario en la plataforma.

4.3 Consultas de Prisma sin paginacion

Los endpoints `GET /api/stores` (lista de tiendas) y `GET /api/store-products` (productos por tienda) no implementan paginacion. Cuando el numero de tiendas crece a cientos o los productos a miles, estas consultas devolveran conjuntos de datos masivos que consumiran memoria excesiva en el servidor y causaran tiempos de carga lentos en el cliente. La ausencia de paginacion tambien afecta la interfaz de administracion, donde la tabla de usuarios, tiendas y productos se vuelve ilegible con mas de 50 registros. La implementacion de paginacion con cursor (basada en el ID del ultimo registro) es mas eficiente que la paginacion por offset para conjuntos de datos grandes.

4.4 Logging de consultas habilitado en produccion

El archivo `src/lib/db.ts` crea el cliente de Prisma con la opcion `log: ["query"]` habilitada. Esto registra absolutamente cada consulta SQL ejecutada en la consola del servidor. En un entorno de produccion con multiples tiendas generando trafico constante, este logging genera un volumen masivo de salida de consola que impacta el rendimiento del servidor y puede filtrar datos sensibles (como los datos personales de los usuarios) en los logs de Vercel. La solucion es habilitar el logging de consultas unicamente en modo desarrollo usando una condicional basada en la variable de entorno `NODE_ENV`.

4.5 Formulario de contacto sin backend

El componente `ContactPage.tsx` muestra un formulario de contacto que, al ser enviado, simplemente cambia el estado local a "enviado" sin realizar ninguna llamada API ni enviar ningun correo electronico. Para los usuarios que visitan la plataforma con intencion de contactar al soporte tecnico o al equipo de ventas, esta implementacion falsa genera frustracion y pierde oportunidades comerciales. La implementacion de un servicio de correo transaccional (como Resend, SendGrid o el servicio de correo de Supabase) es necesaria para que el formulario de contacto funcione realmente.

5. Evaluacion de Componentes

5.1 Componentes bien contruidos

A pesar de los problemas de seguridad y arquitectura, la plataforma tiene componentes de interfaz que demuestran un buen nivel de diseño y calidad visual. Las tres plantillas de tienda (Moderna, Vibrante, Clásica) son responsivas, tienen animaciones fluidas con Framer Motion, y ofrecen una experiencia de compra atractiva con integración de WhatsApp. El componente de vista de producto detallado incluye breadcrumbs, productos relacionados, botón de WhatsApp para contacto directo, badges de destacado y nuevo, y función de compartir en redes sociales. El dashboard de administración ofrece métricas ricas como MRR (Ingresos Mensuales Recurrentes), alertas de facturación, distribución de planes y ranking de tiendas principales. Estas interfaces fueron claramente diseñadas con atención al detalle y ofrecen una base visual sólida para el producto.

5.2 Componentes que requieren mejoras

Componente	Problema	Impacto
ProductForm	Solo acepta URL de imagen, no hay subida de archivos	Alto
PlanManager	Cambio de plan solo actualiza estado local, no llama API de pagos	Critico
StoreSettings	Cambios no persisten al servidor	Alto
AdminPlans	Llama a GET /api/payments pero ese endpoint no devuelve arreglo de pagos	Alto
ContactPage	Formulario no envia datos a ningun backend	Medio
ResetPasswordPage	Flujo completo sin implementar (no hay endpoint)	Critico
StoreQRCode	Depende de API externa sin manejo de errores	Medio

Tabla 2: Componentes con deficiencias identificadas

6. Mapa de Endpoints API

La plataforma expone 15 endpoints API agrupados por funcionalidad. La siguiente tabla resume cada endpoint, su método HTTP, nivel de autenticación requerido y observaciones sobre su estado actual. Esta información es esencial para priorizar las correcciones de seguridad, ya que los endpoints públicos sin autenticación representan la mayor superficie de ataque.

Endpoint	Método	Auth	Observación
/api	GET	Ninguna	Health check - OK

/api/auth	POST	Ninguna	Login - sin rate limiting
/api/auth	GET	Bearer	Verificar token
/api/users	POST	Ninguna	Registro - sin CAPTCHA ni email
/api/users	GET	Admin	Listar usuarios
/api/stores	GET	Mixto	Publico por slug / Admin lista
/api/stores	POST	Auth	Crear tienda
/api/store-products	GET	Publico	Productos por tienda
/api/store-products	POST	Owner	Crear producto
/api/store-products/[id]	DELETE	Owner	Eliminar producto
/api/payments	PUT	Ninguna	Webhook - sin verificacion!
/api/seed	GET	Admin	Destruye y recrea datos
/api/settings	PUT	Admin	En memoria, se pierde al reiniciar
/api/download-zip	GET	Admin	Placeholder no implementado

Tabla 3: Inventario completo de endpoints API

7. Plan de Remediacion Priorizado

A continuacion se presenta el plan de remediacion organizado en cuatro fases, desde los bloqueadores que deben resolverse inmediatamente hasta las mejoras de calidad que pueden implementarse de forma gradual despues del lanzamiento inicial. Cada fase incluye las acciones especificas, los archivos afectados y una estimacion del esfuerzo requerido para su implementacion.

Fase 1: Bloqueantes (Antes de produccion)

Las siguientes acciones son absolutamente obligatorias antes de aceptar cualquier usuario real o dato de produccion. Cada una de estas correcciones previene un escenario catastrofico que podria destruir la credibilidad del negocio o causar perdidas financieras directas.

Accion	Archivo	Esfuerzo
Rotar claves Supabase y JWT	.env, auth.ts	Bajo
Eliminar fallback de secreto JWT	src/lib/auth.ts	Bajo
Verificar firma en webhook de pagos	src/app/api/payments/route.ts	Medio
Eliminar contraseñas de mock-data	src/lib/mock-data.ts	Bajo

Agregar rate limiting en auth	middleware.ts (nuevo)	Medio
Agregar validacion Zod en APIs	Todas las rutas API	Alto
Desactivar logging de queries	src/lib/db.ts	Bajo
Habilitar errores TS en build	next.config.ts	Medio

Tabla 4: Acciones de la Fase 1 - Bloqueantes

Fase 2: Hardening de Seguridad

Estas acciones refuerzan la seguridad del sistema y deben implementarse en las primeras semanas de operacion. Protegen contra vectores de ataque avanzados y mejoran la postura de seguridad general de la plataforma para cuando comience a recibir trafico organico.

Accion	Descripcion
Migrar JWT a cookies httpOnly	Eliminar localStorage, usar cookies seguras con banderas httpOnly y Secure
Implementar CSRF protection	Agregar middleware.ts con verificacion de tokens CSRF para formularios
Sanitizar respuestas de error	Reemplazar err.message con mensajes genericos en respuestas 500
Verificar permisos de suscripcion PUT	Agregar verificacion de que el usuario es dueño de la suscripcion
Implementar reset real de contrasena	Crear flujo con token temporal y envio de email
Agregar verificacion de email	Enviar email de confirmacion al registrar nueva cuenta

Tabla 5: Acciones de la Fase 2 - Hardening

Fase 3: Mejoras Arquitectonicas

Estas mejoras transforman la plataforma de un prototipo funcional en un sistema escalable preparado para cientos de tiendas. Deben planificarse para el primer trimestre de operacion y ejecutarse de forma incremental para no interrumpir el servicio existente.

- **Migrar a PostgreSQL (Supabase):** Reemplazar SQLite con PostgreSQL aprovechando la conexion de Supabase ya configurada en las variables de entorno. Esto habilita concurrencia real, pool de conexiones, y alta disponibilidad.
- **Migrar settings a la base de datos:** Crear modelo PlatformSetting en Prisma y persistir toda la configuracion de la plataforma.
- **Implementar SSR para paginas de tiendas:** Usar file-based routing de Next.js para que cada tienda tenga sus propias rutas con SSR, permitiendo indexacion por Google y

tiempos de carga optimizados.

- **Agregar paginacion en todas las consultas:** Implementar paginacion con cursor en endpoints de listado de tiendas, productos y usuarios.
- **Implementar subida real de imagenes:** Integrar Supabase Storage para permitir a los propietarios subir fotos de productos desde el dashboard.
- **Persistir deleteProduct en el servidor:** Agregar llamada API DELETE cuando se elimina un producto desde el dashboard.
- **Implementar error boundaries:** Agregar error.tsx y global-error.tsx para manejar errores de forma graceful.
- **Agregar tests automatizados:** Configurar Vitest y Testing Library para pruebas unitarias de API routes y componentes criticos.

Fase 4: Calidad y Optimizacion

Las siguientes acciones son mejoras de calidad que pueden implementarse de forma continua despues del lanzamiento. No son bloqueantes pero contribuyen significativamente a la estabilidad a largo plazo y a la experiencia profesional de la plataforma.

- **Habilitar reactStrictMode:** Re-activar el modo estricto de React para detectar efectos secundarios indeseados y problemas de renderizado.
- **Completar metadatos SEO:** Agregar favicon.ico, og-image.png, apple-touch-icon.png y el codigo real de verificacion de Google.
- **Implementar monitoreo:** Integrar Sentry para captura automatica de errores en produccion y alertas en tiempo real.
- **Re-habilitar reglas de ESLint:** Restaurar reglas de linting progresivamente para mantener la calidad del codigo.
- **Agregar soft delete:** Implementar eliminacion suave con campo "deletedAt" para permitir recuperacion de datos eliminados accidentalmente.
- **Agregar auditoria de acciones:** Registrar un log de quien realizo cada accion critica (cambios de plan, eliminaciones, cambios de configuracion).
- **Limpiar repositorio:** Eliminar directorios tienda-app/ y skills/ del repositorio principal para evitar confusion.

8. Conclusion

TiendApp posee una base funcional notable con componentes de interfaz bien disenados, una estructura de datos coherente con 6 modelos en Prisma, y una vision de producto clara

orientada al mercado peruano. La plataforma ofrece funcionalidades atractivas como la creacion guiada de tiendas mediante un wizard, tres plantillas de diseno profesional, integracion con WhatsApp, y un sistema de planes de suscripcion Free, Pro y Premium que sigue un modelo de negocio SaaS valido.

Sin embargo, la plataforma **no esta lista para albergar multiples tiendas en un entorno de produccion** en su estado actual. Las seis vulnerabilidades criticas de seguridad descritas en la seccion 2 representan riesgos que podrian comprometer completamente el sistema si son explotadas antes de ser corregidas. El uso de SQLite como base de datos, la arquitectura SPA que anula las ventajas de Next.js, y la falta de paginacion en las consultas limitan severamente la capacidad de la plataforma para escalar. Los problemas de consistencia de datos (como deleteProduct que no persiste al servidor) erosionarian rapidamente la confianza de los propietarios de tiendas en la herramienta.

La recomendacion es ejecutar las acciones de la Fase 1 (bloqueantes) de forma inmediata, lo cual requiere un esfuerzo estimado de 2 a 3 semanas de desarrollo. Las Fases 2 y 3 deben completarse durante el primer trimestre de operacion, mientras que la Fase 4 puede ejecutarse de forma continua. Con estas correcciones implementadas, TiendApp tendria los cimientos tecnicos necesarios para operar como un negocio de plataformas de comercio electronico escalable y confiable en el mercado peruano.